EMBEDFACTORY · MAPPA DEL SISTEMA · CONTROL PLANE V1

Dal requisito al **firmware verificato**.

Una software factory conversazionale per firmware embedded: scheletro deterministico → patch validate → build reale → riparazione guidata dagli errori del compilatore → validazione dell'artefatto. Zephyr è l'implementazione completa oggi; FreeRTOS è un modello di stato già previsto, senza finta prontezza.

`VERIFICATION · REAL / SIMULATED / NOT_AVAILABLE` `VERDICT · SUCCESS / FAILURE / UNKNOWN` `NESSUN READY SENZA EVIDENZA`

01 Cosa risolve

IL PROBLEMA È LA FIDUCIA, NON LA GENERAZIONE

PROBLEMA

Firmware "scritto bene" che non è mai stato compilato

Un LLM può produrre codice plausibile e totalmente non compilabile: nessuno se ne accorge finché non tocca il toolchain.

EmbedFactory: ogni modifica passa da un build reale; l'output del compilatore è l'unica verità.

PROBLEMA

Setup manuale di skeleton e board

CMakeLists, prj.conf, main.c, board name: lavoro ripetitivo e fonte di errori silenziosi a ogni nuovo progetto.

EmbedFactory: skeleton deterministico, versionato, con board scelta solo tra quelle che il doctor ha davvero elencato.

PROBLEMA

Nessuna traccia di chi ha cambiato cosa

Patch applicate "a memoria", stato del progetto indistinguibile tra "sembra pronto" e "è stato costruito".

EmbedFactory: versioni immutabili con patch, autore, motivo e verdetto di build; ripristino di qualsiasi versione.

PROBLEMA

Non si sa cosa manca per poter buildare

Errori generici, toolchain assenti, runner spento: l'utente non sa se il problema è il codice o l'ambiente.

EmbedFactory: doctor reale + control plane che dice *cosa* non è pronto e *perché*.

02 Input

TUTTO CIÒ CHE ENTRA NEL SISTEMA

IN Requisito e contesto

REQUISITO IN LINGUAGGIO NATURALE

Cosa deve fare il firmware, in italiano o inglese, anche in più turni conversazionali.

es. `"LED1 a 1 Hz su nucleo_l476rg, nessun delay bloccante nell'ISR"`

RTOS

Zephyr (implementazione completa) o FreeRTOS (stato modellato, nessun doctor reale: `NOT_CONFIGURED`).

BOARD / MCU

Identificatore validato. Diventa `READY` solo se compare nella lista reale del doctor, altrimenti `UNKNOWN / NOT VERIFIED`.

VINCOLI E CONOSCENZA

Regole di sicurezza embedded (allocazione statica, ISR brevi, API FromISR) + frammenti di knowledge base con budget.

CONFIGURAZIONE SERVER-SIDE

Provider e modello LLM, endpoint del build runner. I secret restano nel processo, mai nel browser.

`LLM_PROVIDER · LLM_BASE_URL · LLM_MODEL · LLM_API_KEY · BUILD_RUNNER_URL · BUILD_RUNNER_TOKEN`

CONTESTO Cosa legge già dentro

FILE CORRENTI DEL PROGETTO

Ogni turno riceve il contenuto attuale: il modello non lavora su una copia mentale.

ULTIMO BUILD REALE

Comando, exit code, stdout/stderr del compilatore e artefatti: è l'input della riparazione.

PATCH PRECEDENTI

Evita cicli che riapplicano la stessa correzione fallita.

DOCTOR DELL'AMBIENTE

west, cmake, ninja, python, dtc, ZEPHYR_BASE, Zephyr SDK, arm-zephyr-eabi, board list. Nessuna inferenza.

IDENTITÀ E PROPRIETÀ

Sessione Convex Auth: ogni query/mutation verifica la proprietà lato server. Nessun userId dal client.

03 Come funziona — la pipeline

UN TURNO, SETTE PASSAGGI VERIFICABILI

01 Skeleton `DETERMINISTICO · NESSUN LLM`

Il progetto nasce da template fissi: `CMakeLists.txt`, `prj.conf`, `src/main.c`, `README.md`, `.gitignore`, con board e RTOS richiesti. La prima versione è sempre ricostruibile e non dipende da un modello linguistico.

```
IN   nome, rtos, board, requisito
→
OUT  albero file + versione v1
→
```

02 Contesto e prompt `SERVER-SIDE`

Retrieval della knowledge base entro un budget, poi un prompt strutturato: progetto, RTOS, board, MCU, file correnti, patch già applicate, ultimo build reale e richiesta utente.

```
IN   requisito + stato progetto
→
OUT  prompt completo (tracciato)
→
```

03 Piano del modello `UN SOLO OGGETTO JSON`

Il modello non risponde a testo libero: restituisce `{ summary, creates, patches, requestBuild, answerOnly }`. La risposta viene estratta e validata; se il provider è irraggiungibile il turno fallisce esplicitamente: nessun file creato, nessun file modificato, nessun build avviato.

```
IN   prompt
→
OUT  piano di patch o answer-only
→
ERR  LLM_BAD_RESPONSE · PLAN_EMPTY
→
```

04 Validazione patch `TUTTO-O-NIENTE`

I file esistenti si modificano solo con unified diff a 3 righe di contesto: se il contesto non combacia byte per byte l'intera operazione è rifiutata (`PATCH_FAILED`) e non viene scritto nulla. I path sono confinati a `src/`, `include/`, `drivers/`, `app/`, `tests/`, `boards/`, `scripts/` e ai file radice ammessi.

```
IN   piano + contenuto corrente
→
OUT  file aggiornati + versione immutabile
→
```

05 Build reale `IL RUNNER È L'UNICO CHE COMPILA`

Dispatch HTTP al runner, che esegue davvero `west build -b <board> -d build -p auto` nella working directory corretta. Torna exit code, comando, stdout/stderr, artefatti e memoria linker. Convex non può lanciare processi: senza runner il risultato è `NOT_AVAILABLE`, mai un build finto.

```
IN   progetto + board + toolchain
→
OUT  REAL · exit 0
→
OUT  REAL · errore compilatore
→
```

06 Ciclo di riparazione `MAX 3 TENTATIVI`

In caso di `FAILURE` gli errori reali di GCC/linker rientrano nel prompt con l'istruzione di correggere solo quei punti, non di riscrivere il file. Ogni tentativo è un build reale. Alla fine: `BUILD FAILED` o `REPAIR LIMIT REACHED`, con la cronologia completa.

```
IN   stderr + exit code del build
→
OUT  patch correttiva → rebuild
→
```

07 Validazione artefatto `L'EVIDENZA`

L'ELF viene ispezionato senza dipendenze esterne: magic number, header ELF32/64, endianness, architettura (ARM, RISC-V, Xtensa, ARC), entry point, dimensione e SHA-256. La memoria FLASH/RAM arriva dal report del linker. Il manifest di export porta con sé board, toolchain e verdetto.

```
IN   zephyr.elf + report linker
→
OUT  ELF ARM · sha256 · FLASH/RAM
→
```

04 Output

COSA RESTA ALL'UTENTE

OUT Artefatti del prodotto

PROGETTO FIRMWARE VERSIONATO

File reali, più la storia `v1…vn`: contenuto, autore, motivo, patch e verdetto di build. Ogni versione è ripristinabile.

EVIDENZA DI BUILD REALE

Comando, exit code, durata, stdout/stderr e artefatto ispezionato.

es. `zephyr.elf · ELF · ARM · sha256:… · FLASH 41 236 B / 256 KB · RAM 9 536 B / 64 KB`
valori emessi dal runner su una macchina con toolchain

MANIFEST + EXPORT ZIP

ZIP costruito davvero nel browser (file + README + manifest) e inizializzazione Git opzionale.

`{ rtos, board, toolchain, verification, verdict }`

STATO ONESTO DEL PROGETTO

`verified` solo dopo un build reale riuscito; altrimenti `unverified`. Le righe del vecchio motore simulato restano `LEGACY · UNVERIFIED`.

NON-OUT Quello che non produce (per scelta)

NESSUN "DEPLOY" O FLASH SULLA BOARD

HIL, flashing e test su hardware non sono implementati: non esiste una UI che li simuli.

NESSUN VERDETTO SENZA ESECUZIONE

Un runner che risponde "successo" con exit code diverso da 0 viene declassato a `UNKNOWN`.

NESSUNA IMPORTAZIONE DI FIRMWARE ESISTENTE

L'entry point "Existing Firmware" è visibile ma disabilitato: *Coming soon*, nessuna schermata finta.

NESSUNA BOARD "SUPPORTATA" PER DECRETO

La lista board viene dal doctor. Se non è verificabile, la UI scrive `UNKNOWN / NOT VERIFIED`.

05 Control plane

TRE LIVELLI, UNA SOLA FONTE DI VERITÀ

LIVELLO 1 · ORIENTAMENTO

Dashboard

- **Welcome** + due ingressi: **New Firmware** e **Existing Firmware** (disabilitato).
- **System status** a colpo d'occhio: AI, build runner, Zephyr, FreeRTOS, toolchain.
- **First-run checklist** non bloccante: collega AI → verifica runner → toolchain Zephyr → FreeRTOS.
- Ogni riga di stato apre il dettaglio in Environment.

`/dashboard`

LIVELLO 2 · DIAGNOSI

Environment

- AI: provider, modello, connessione, ultimo controllo reale.
- Runner: URL, stato, latenza, tool riportati da **GET /health**.
- Zephyr: tabella doctor (west, cmake, ninja, python, dtc, ZEPHYR_BASE, SDK, arm-zephyr-eabi, board) e output grezzo.
- FreeRTOS: **Not configured** finché non esiste un doctor reale.
- **Run diagnostics** esegue davvero il doctor via runner.

`/environment`

LIVELLO 3 · CONFIGURAZIONE

Settings

- AI/LLM: sette variabili, più **Test connection** con una generazione reale.
- Build runner: URL e token, più **Test runner** con **GET /health**.
- Secret mostrati solo come *Configured* / *Not configured*.
- General: modelli e limiti già esistenti, nessuna impostazione duplicata.

`/settings`

STATO	vocabolario unico usato da tutte le superfici
AI	● Connected (risposta reale) · ● Not configured · ● Error con messaggio del provider
BUILD RUNNER	● Connected · ● Not configured · ● Unavailable · ● Authentication failed (401/403)
ZEPHYR	● Ready solo se ogni check richiesto è PASS · ● Not ready con l'elenco dei mancanti · ● Unknown senza prove
FREERTOS	● Not configured — nessun doctor reale, nessuna prontezza dichiarata
TOOLCHAIN	● Ready · ● Not ready · ● Unknown
BOARD	● Ready se `west boards` l'ha elencata · ● UNKNOWN / NOT VERIFIED altrimenti

06 La regola di onestà

DUE CAMPI, NESSUNA ECCEZIONE

VERIFICATION	EXIT CODE	VERDICT	COSA VEDE L'UTENTE
REAL	0	SUCCESS	Build eseguito, artefatto validato, progetto *verified*.
REAL	≠ 0	FAILURE	Errori del compilatore reali + tentativi di riparazione.
SIMULATED	—	UNKNOWN	Eredità del motore rimosso: *LEGACY · UNVERIFIED*.
NOT_AVAILABLE	—	UNKNOWN	*reason: toolchain unavailable — "west" is not installed or not in PATH*

- **SUCCESS** esiste solo con `REAL` e `exitCode = 0`.
- Un progetto diventa **verified** solo dopo un build reale riuscito.
- Un probe memorizzato viene **invalidato** appena cambia la configurazione che ha misurato.
- Secret: **Configured / Not configured**. Mai in chiaro, mai in una risposta API, mai in un log, mai in `localStorage`.
- Se manca un pezzo, la UI dice **cosa** e **perché**; non riempie il vuoto con un badge verde.

> "Dimmi cosa è pronto, cosa non lo è, e perché."

07 Architettura

CHI PUÒ ESEGUIRE COSA

CLIENT

Web · React 19 + Vite

Legge solo stato normalizzato. Nessun secret, nessun toolchain, nessun build.

→

BACKEND

Convex · auth + dati

Query/mutation con proprietà verificata, azioni node per LLM e dispatch. `process.env` qui.

→

ESECUZIONE

Runner · Bun HTTP

Unico componente che esegue `west`, `cmake`, `ninja`, `dtc`. Endpoint: health, doctor, build, git, analyze, export.

→

VERITÀ

Toolchain nativa

Zephyr SDK, `arm-zephyr-eabi`, board di `west boards`. L'unica fonte di un SUCCESS.

Perché il runner esiste: un runtime serverless non può avviare processi nativi. Invece di simulare un build, il sistema sposta l'esecuzione dove il toolchain vive davvero e degrada in modo esplicito: senza runner → `NOT_AVAILABLE`; senza doctor → `UNKNOWN`; il progetto esportato resta comunque buildabile in locale.

```
bun runner/index.ts doctor — questa macchina può buildare firmware?
bun runner/index.ts verify --board nucleo_l476rg — fallimento → riparazione → successo, reale
bun runner/index.ts serve --port 8790 — il runner che alimenta la web app
bun test · bunx tsc -b --noEmit · bunx convex dev --once
```

EMBEDFACTORY
REQUIREMENT → VERIFIED FIRMWARE
NESSUN READY SENZA EVIDENZA